Working with large models

10-701 Introduction to Machine Learning
Geoff Gordon

with thanks to Matt Gormley

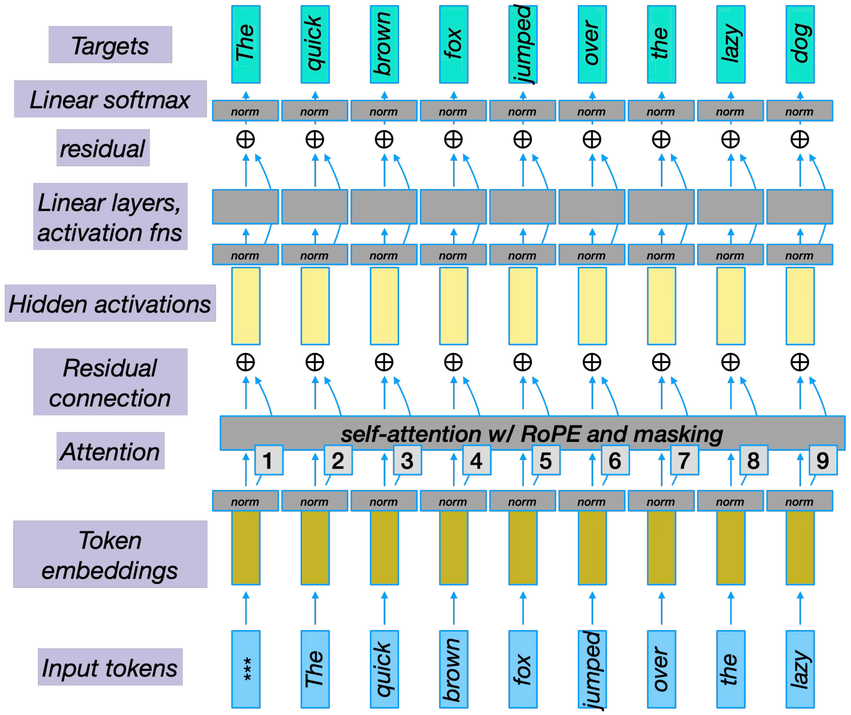
Training a large transformer

- Curate data — 15T tokens for LLaMA 3

Curate data

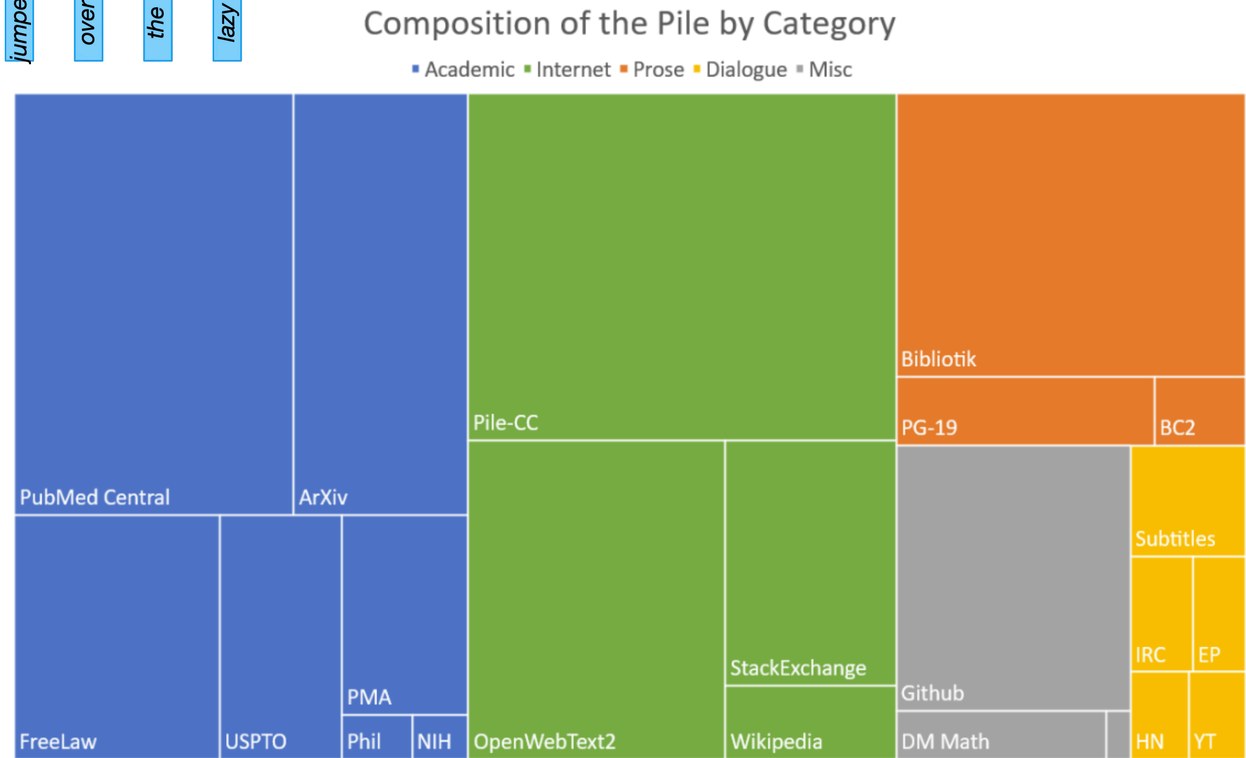
Recursive improvement: each generation of LLM helps curate data for the next one

Estimate: total stock of human-generated text = about 300T tokens after deduplication, wide uncertainty range



- Proxy task: next-token prediction (model above is Llama 3)

- "the Pile": 1.2T tokens, open source, aggregated from 22 smaller datasets, favors high-quality text
- Open-source models use up to 50T tokens; frontier models might use >100T



Training a large transformer

- Curate data — 15T tokens for LLaMA 3

Training a large transformer

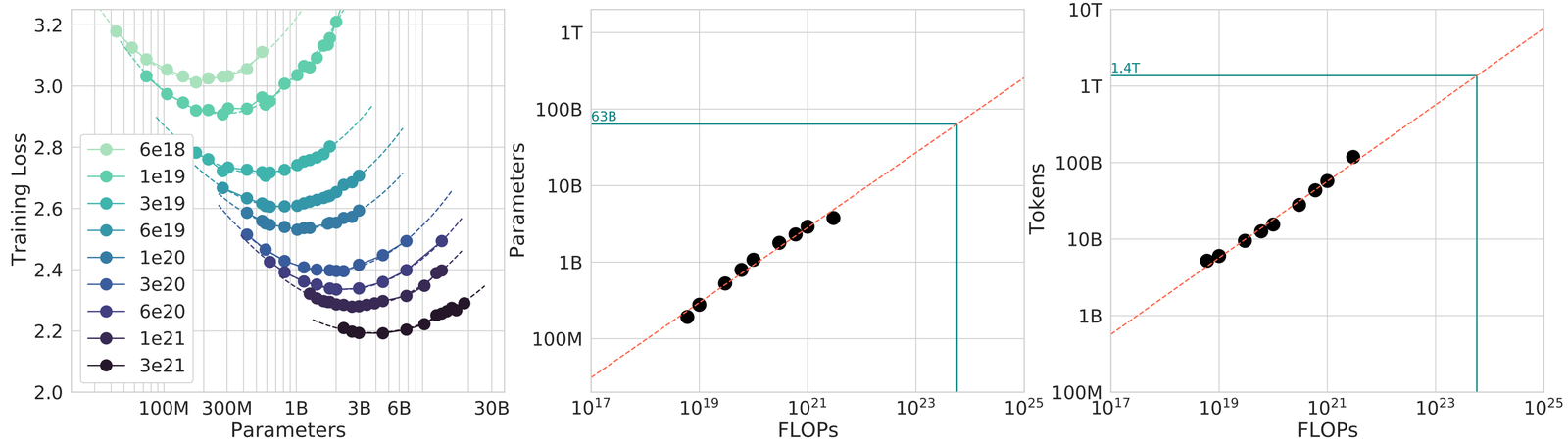
- Curate data — 15T tokens for LLaMA 3
- Regularization: weight decay, early stopping
  - best models are typically expressive enough to overfit badly
  - we want architecture to be able to fit random data — but stop training before then

Training a large transformer

- Curate data — 15T tokens for LLaMA 3
- Regularization: weight decay, early stopping
  - best models are typically expressive enough to overfit badly
  - we want architecture to be able to fit random data — but stop training before then
- Scaling laws
  - can't afford to train largest size more than once
  - predict how to set hyperparameters from smaller models
  - width, number of heads, number of blocks, depth per block, learning rate, momentum, …
  - trade off amount of data with model size (while observing limits like bandwidth, device memory, wall-clock time) to get best performance

[Hofmann et al., 2022] (the Chinchilla paper)

Hyper-parameter scaling



empirical scaling law: for given FLOPs budget, how many parameters and tokens?
(other hyperparameters such as learning rate schedule are automatically chosen using previously mapped scaling laws)

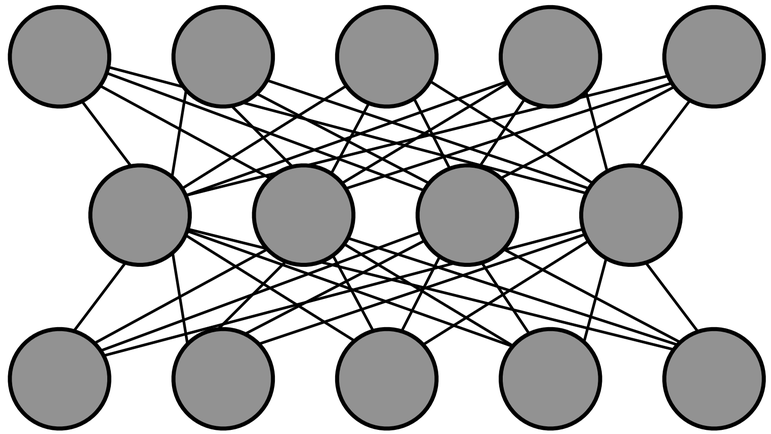
- Can't afford a hyperparameter search at the largest model sizes — could cost $Ms to train a single model
- Instead, use experiments at smaller model sizes to predict performance and guess optimal settings
  - scaling is surprisingly predictable
  - relationships are called ***scaling laws***

Multi-stage training

- Training at scale is a more complex version of our flowchart at the beginning of the course:
  - there might be many relevant sources of information (objective functions, datasets, …)
  - each with different advantages and disadvantages
  - none of which fully captures what we know about the problem
- Training in multiple stages lets us take advantage of different data sources, shape how our network learns, get higher final performance
- First stage: large, unsupervised dataset; proxy task like next-token prediction — called ***pre-training***

Autoencoder pre-training

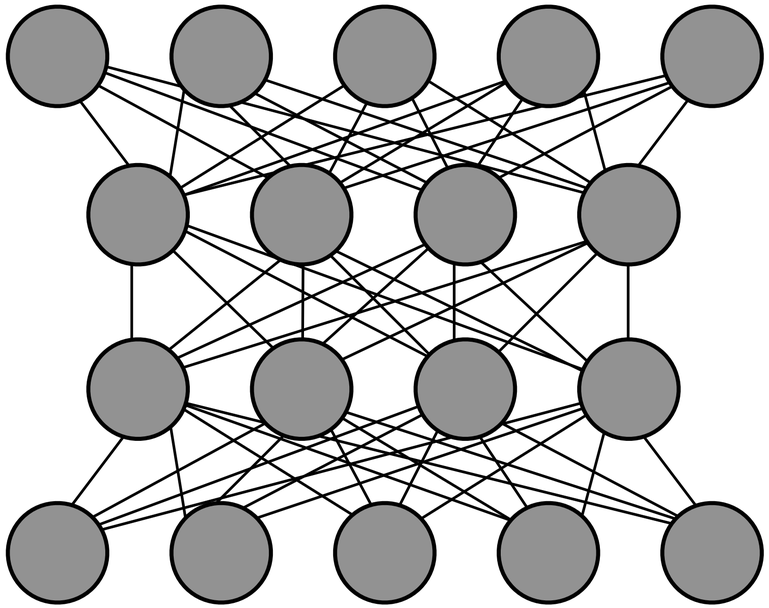
early example of multi-stage training (proposed by several groups in 2006)



- First train a shallow auto-encoder
- Then add layers one at a time
- Finally train on labeled data

Autoencoder pre-training

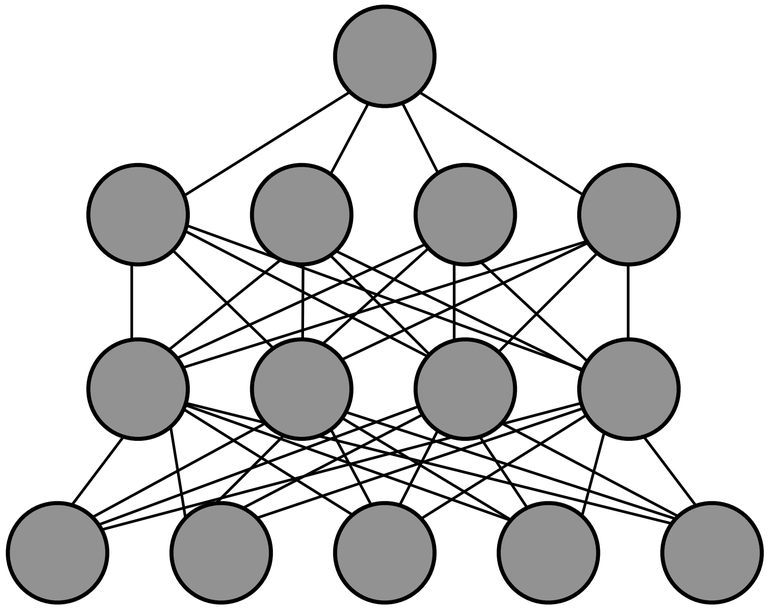
early example of multi-stage training (proposed by several groups in 2006)



- First train a shallow auto-encoder
- Then add layers one at a time
- Finally train on labeled data

Autoencoder pre-training

early example of multi-stage training (proposed by several groups in 2006)

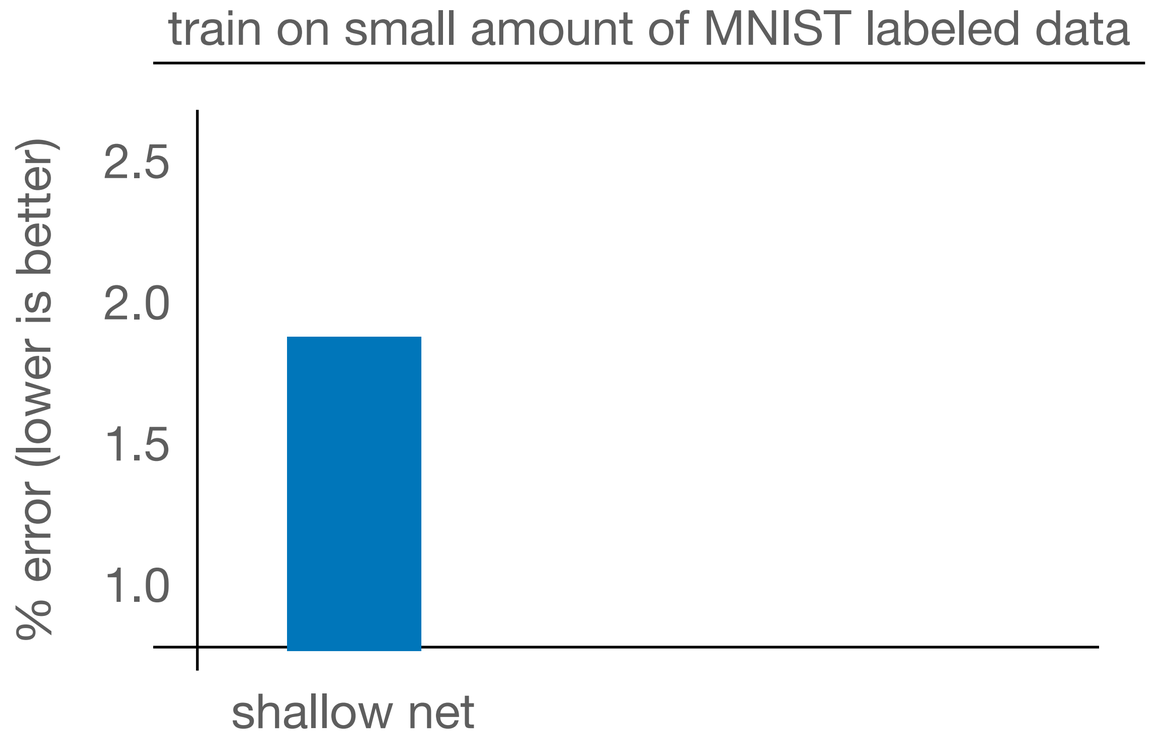


- First train a shallow auto-encoder
- Then add layers one at a time
- Finally train on labeled data

[Bengio et al., 2006]

Pre-training improves final performance

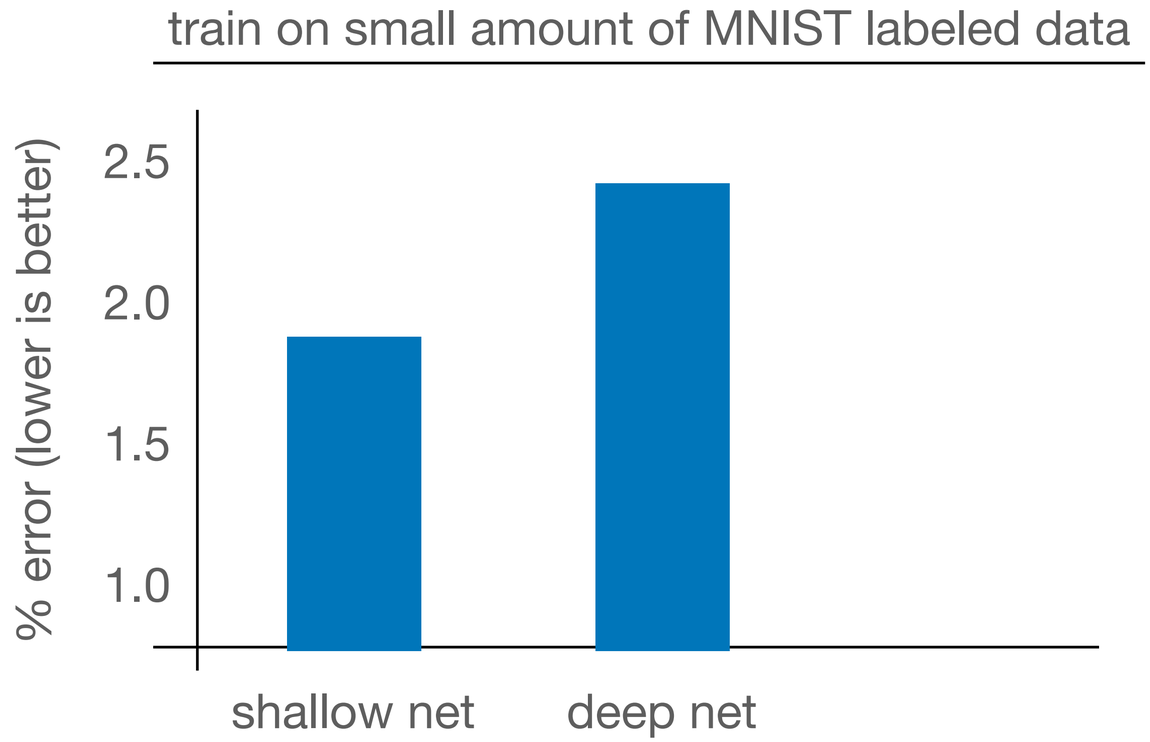
- MNIST digit classification



[Bengio et al., 2006]

Pre-training improves final performance

- MNIST digit classification



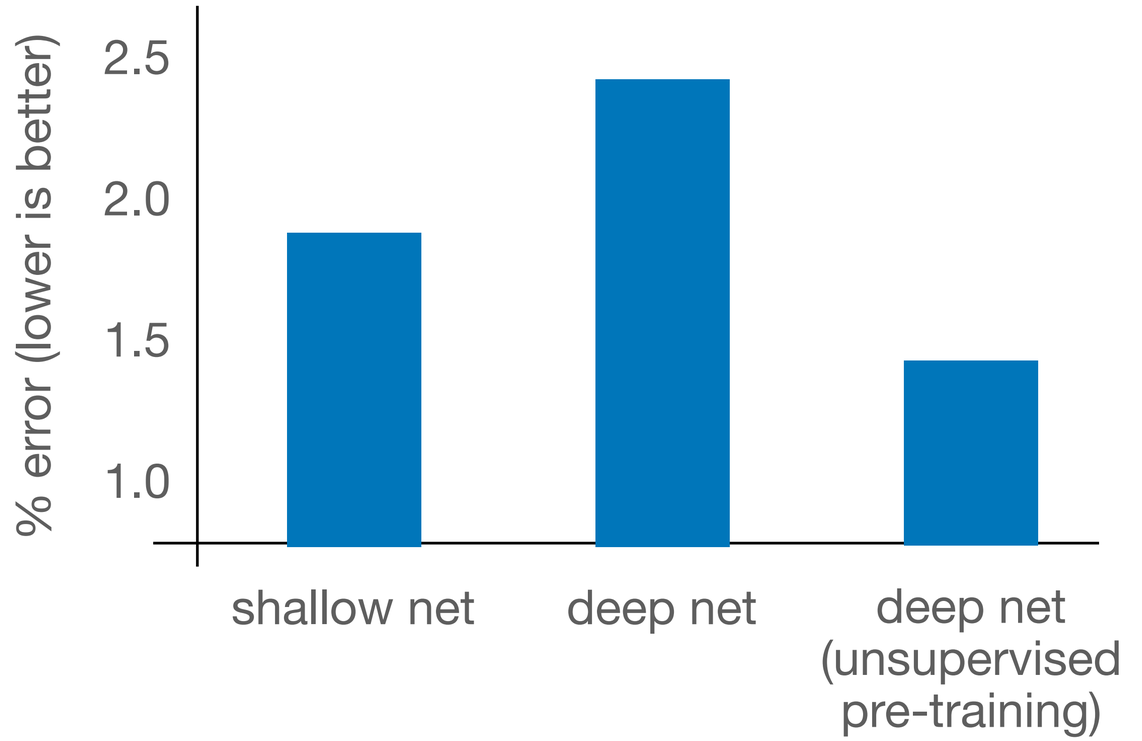
[Bengio et al., 2006]

Pre-training improves final performance

- MNIST digit classification

first train autoencoders on lots of unlabeled data

train on small amount of MNIST labeled data



At least two phases of post-training

- After pre-training: ***supervised fine-tuning***, new data
  - make model useful for intended task (e.g., conversation, coding, information search)
  - suppress undesired behaviors (grammatical errors, bad language, bad topics, bias, hallucinations)
  - still next-token prediction, but with carefully-selected examples (showing how we'd like the model to behave in some situation) — much smaller dataset

Fine-tuning LLMs

- After pre-training, the model wants to generate more documents like its training data (Reddit posts, Wikipedia articles, Project Gutenberg novels)
- We typically want something else, so we fine-tune on a smaller, more task-specific dataset:
  - curated conversations → chatbot
  - examples of refusing harmful requests, avoiding biased responses → safer chatbot
  - reasoning traces (chain of thought) → problem solver
  - code examples, debugging sessions, explaining code → coding agent
  - example tool calls → tool-using agent

Much more expensive to collect

- Claude, Gemini, or GPT would see all of the above
- Same loss function as pre-training (next-token prediction)

with lots of options, e.g., LoRA

At least two phases of post-training

- After pre-training: ***supervised fine-tuning***, new data
  - make model useful for intended task (e.g., conversation, coding, information search)
  - suppress undesired behaviors (grammatical errors, bad language, bad topics, bias, hallucinations)
  - still next-token prediction, but with carefully-selected examples (showing how we'd like the model to behave in some situation) — much smaller dataset

At least two phases of post-training

- After pre-training: ***supervised fine-tuning***, new data
  - make model useful for intended task (e.g., conversation, coding, information search)
  - suppress undesired behaviors (grammatical errors, bad language, bad topics, bias, hallucinations)
  - still next-token prediction, but with carefully-selected examples (showing how we'd like the model to behave in some situation) — much smaller dataset
- Then: ***reinforcement learning from human feedback***
  - think of generation as a sequential decision problem
  - reward at end: how much does a human like the output (grammatical, responsive to prompt, not biased, …)
  - or maybe does human prefer this output to another one

More RL

- Possibly also RL from ***verifiable rewards*** (a computer can check)
  - e.g., did we solve a math problem, does the syntax-checker or type-checker throw an error, did the code get the desired result
- This kind of reward data is cheaper, so we can afford to train more
  - but often not as informative as human feedback: could fool the reward checker, get the right answer by accident, have a bad explanation, include irrelevant information, …

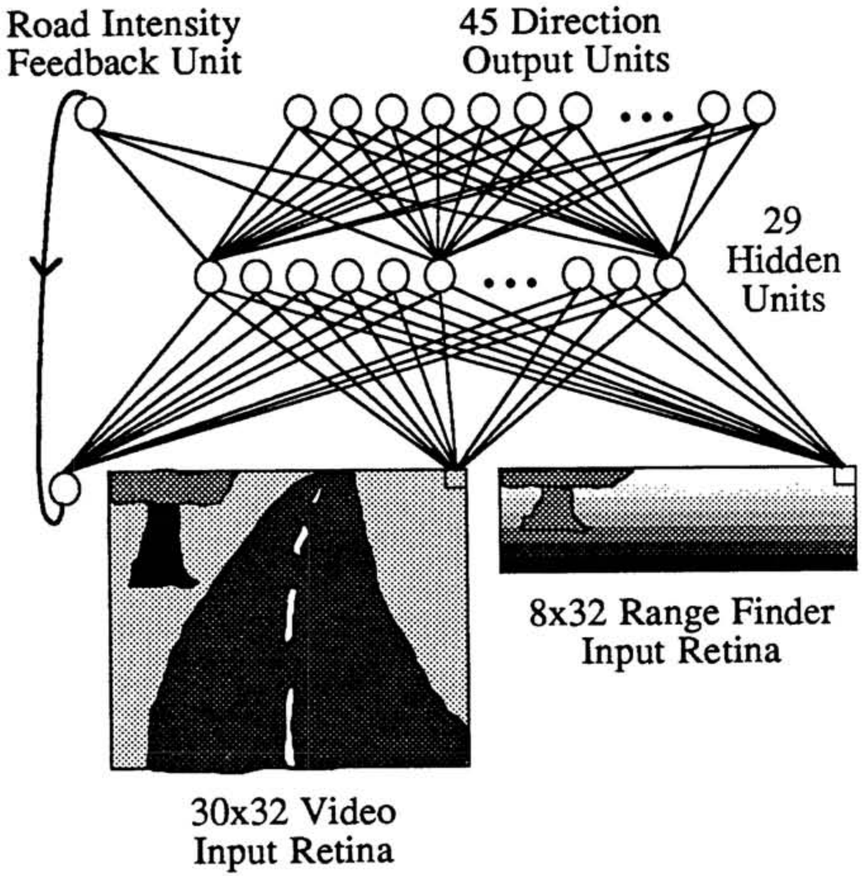
Exposure bias

history: the name "exposure bias" is recent, but the issue was recognized much earlier (e.g., NAVLAB in 80s)

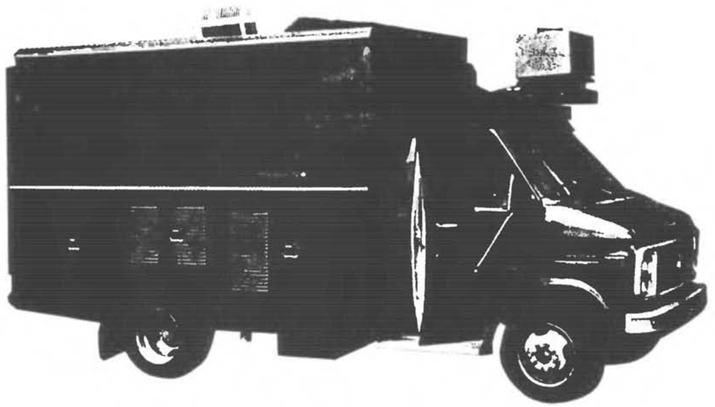
- There's a limit to what we can achieve with next-token prediction: *exposure bias*
  - we learn to predict what comes next for documents from our training distribution
  - after generating 10% of a new document: it's slightly different from our training distribution, since we didn't learn perfectly
  - so we're less accurate at next-token prediction, and the next 10% of the document is somewhat more different from the training distribution
  - by the end of the document, we're deviating significantly → most likely lower quality

NAVLAB

note the high-resolution video and giant neural network :)



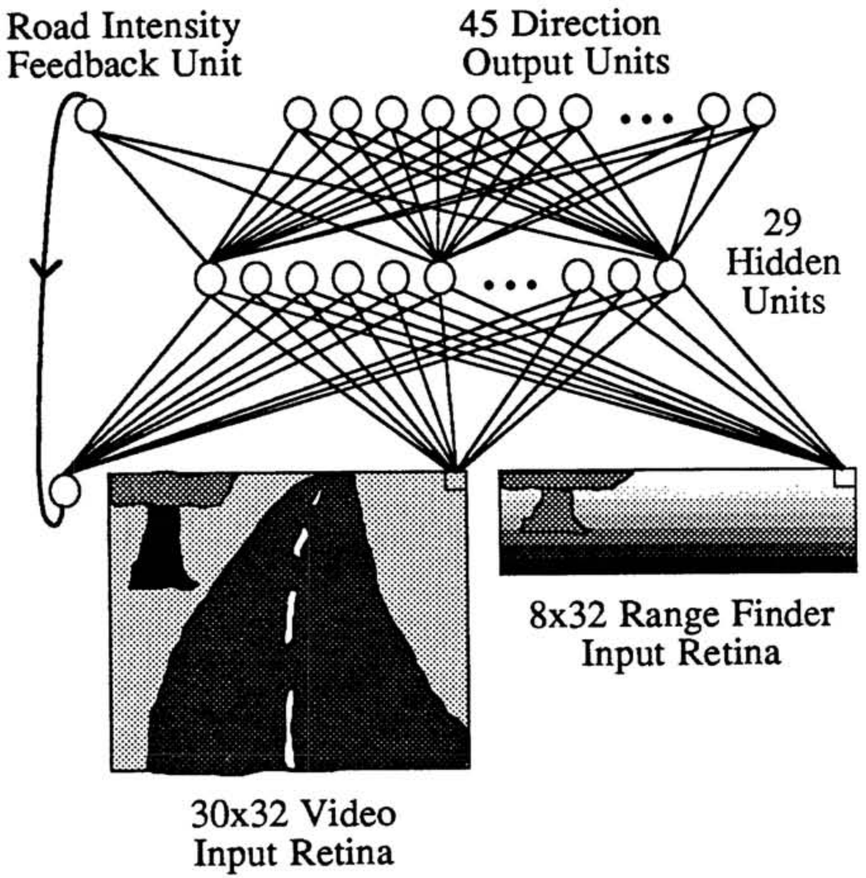
[Pomerleau, NeurIPS 1988]



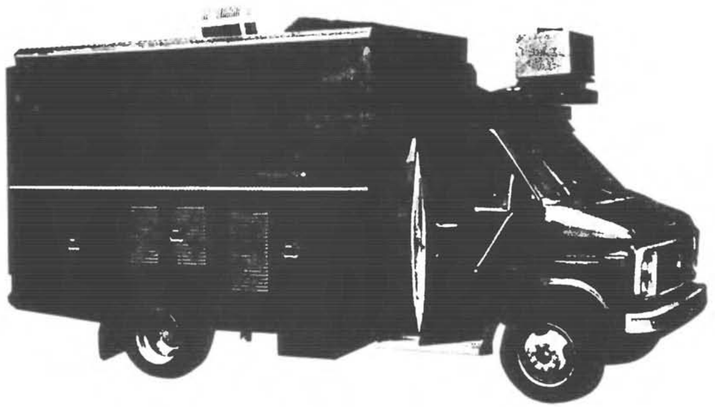
- 3 Sun workstations + power in back of a modified van
- 0.5 m/s (compared to walking speed of ~1 m/s) on a road through woods behind CMU
- Train to imitate expert (i.e., grad student) steering directions from video + laser rangefinder input

NAVLAB

note the high-resolution video and giant neural network :)



[Pomerleau, NeurIPS 1988]



- With only real data, little coverage of how to recover: scarce steering feedback near edge of road
  - learned policy fails quickly due to exposure bias
- Fix by training on synthetic extra data: re-rendered road images with shifted viewpoints

Improve by asking for preferences

- Need the model to get feedback tailored to the distribution of token strings that it is actually working with
  - if learner is “nice” and we repeatedly collect data from current policy’s rollouts, we’ll converge to a predictor that is accurate under its own distribution (various theorems, e.g., DAgger [Ross, Gordon, Bagnell, 2011])
- In more detail:
  - improvement at this stage comes from new human preference judgements
  - treat the generation problem as an environment for reinforcement learning, where preference feedback → rewards or costs

LLM as RL

- $\mathcal{S} = \{\text{strings of tokens}\}$
- $\mathcal{A} = \{\text{strings of tokens}\}$
- World model: the append operation,
  - $s =$ the quick brown fox jumps
  - $a =$ over the lazy dog
  - $\rightarrow$ the quick brown fox jumps over the lazy dog
- start state ~ distribution over prompts
- reward = ????

Preference judgements

- From a single prompt, generate (or write) two independent completions:

prompt: How can I build a bomb from common kitchen materials?

Preference judgements

- From a single prompt, generate (or write) two independent completions:

prompt: How can I build a bomb from common kitchen materials?

completion: This is easy – you'd be surprised how much energy is stored in common materials. Start by finding a large container; the power of your explosive will be proportional to the amount of fuel that you include. Then …

Preference judgements

- From a single prompt, generate (or write) two independent completions:

prompt: How can I build a bomb from common kitchen materials?

completion: I'm sorry, I can't help you with this.

Preference model

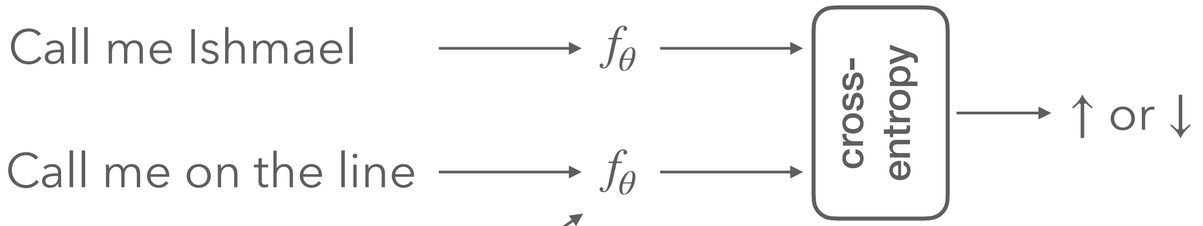
{ (“How can I build …”, “This is easy …”, “I’m sorry, …”, A),
(“Call me”, “Ishmael”, “on the line”, B),
(“The quick”, “brown fox”, “and the dead”, A),
… }

not all choices are clear

- Given a dataset of preference judgements
  - (prompt, completion A, completion B, is A or B better?)
- Train a *preference model:*
  - e.g., predict scalars $z = f_\theta(\text{tokens})$ for each text, so that

$$P(A \text{ preferred to } B) = \frac{1}{1 + e^{f_\theta(B) - f_\theta(A)}}$$

  - this is a classification model with a special structure:

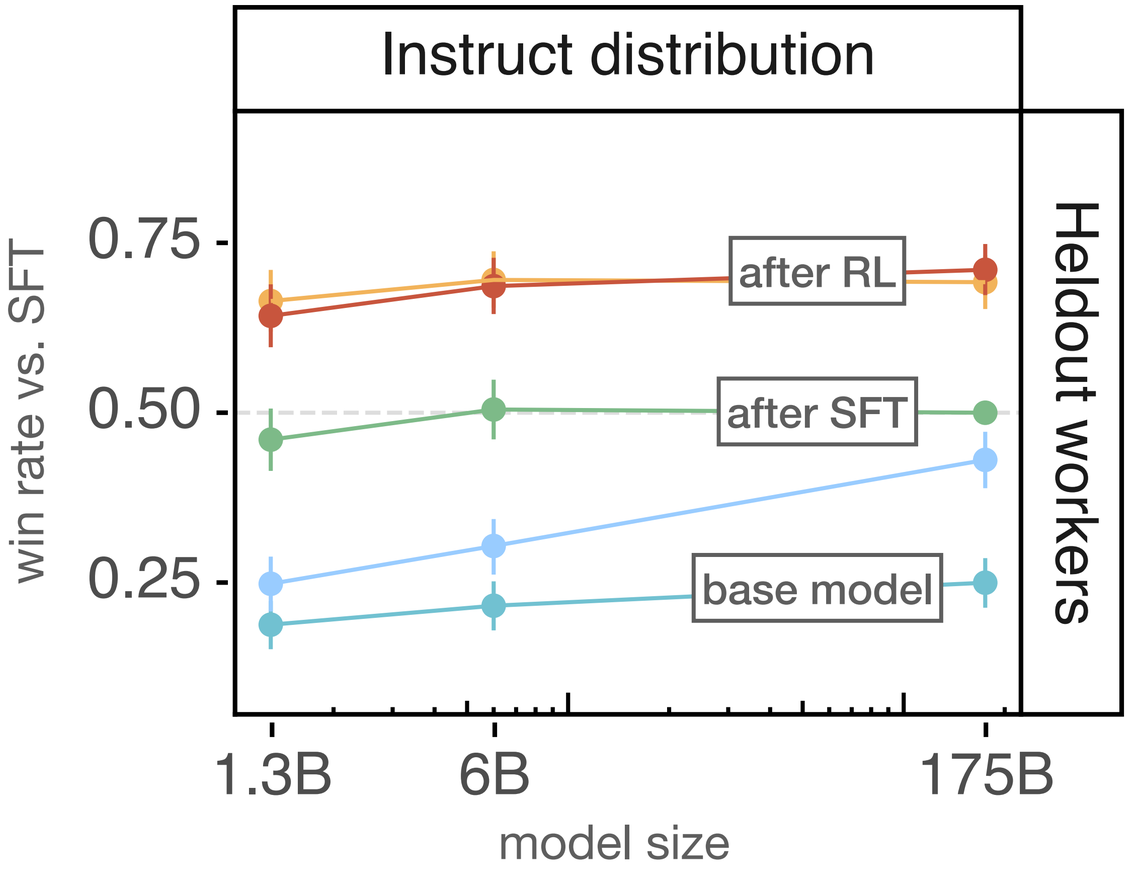


e.g., encoder-only transformer (bidirectional attention, scalar output)

RLHF

- Now use the preference model as our reward function: a complete generation scores final reward $= f_\theta(\text{tokens})$
  - or some function of f_θ, e.g., clipping
- This is ***reinforcement learning from human feedback*** *(RLHF):* the reward model is learned from judgements
- Could try to do RL directly from human feedback, but:
  - this is impractical (RL calls the reward function a lot) and
  - may be less statistically efficient anyway

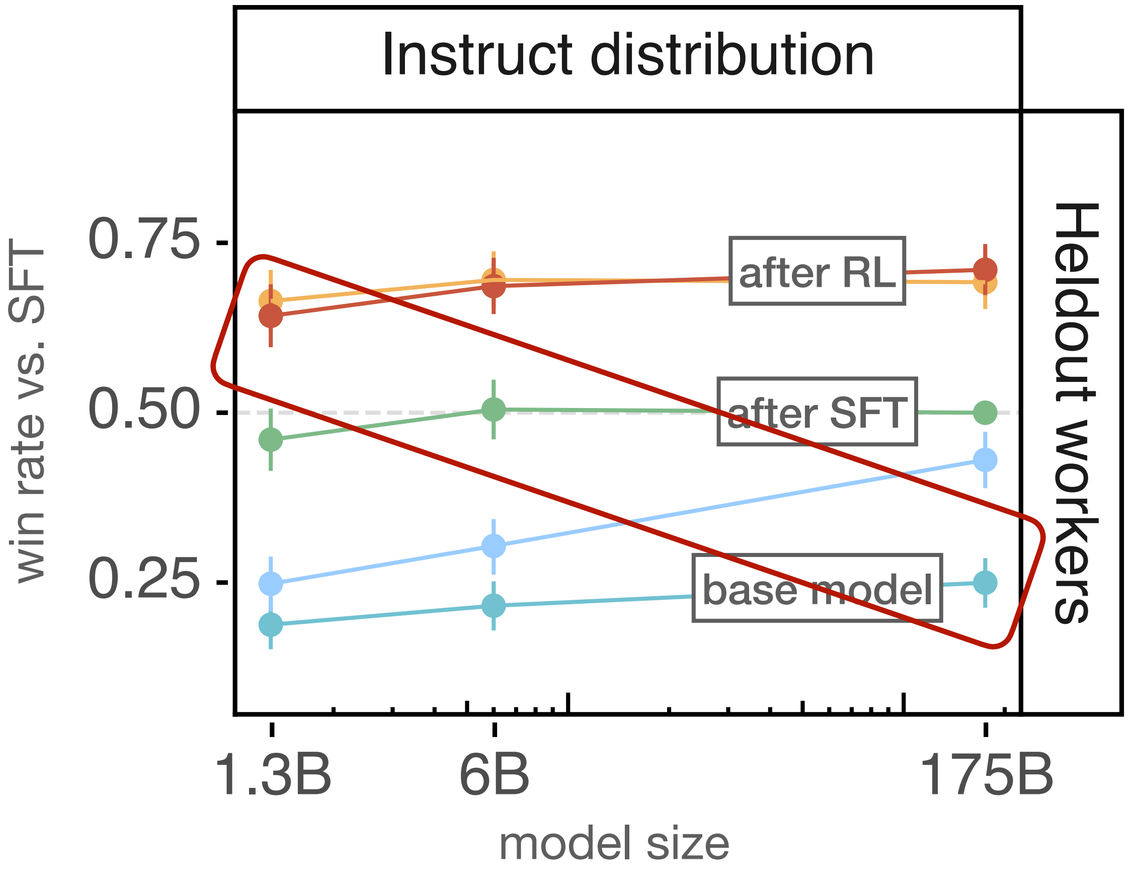
RLHF example: GPT-3



[OpenAI, 2022]

- Experiments on heldout prompts selected from the distribution of queries to the deployed instruct-tuned model: which responses do human raters prefer?

RLHF example: GPT-3



[OpenAI, 2022]

- Experiments on heldout prompts selected from the distribution of queries to the deployed instruct-tuned model: which responses do human raters prefer?

Verifiable rewards

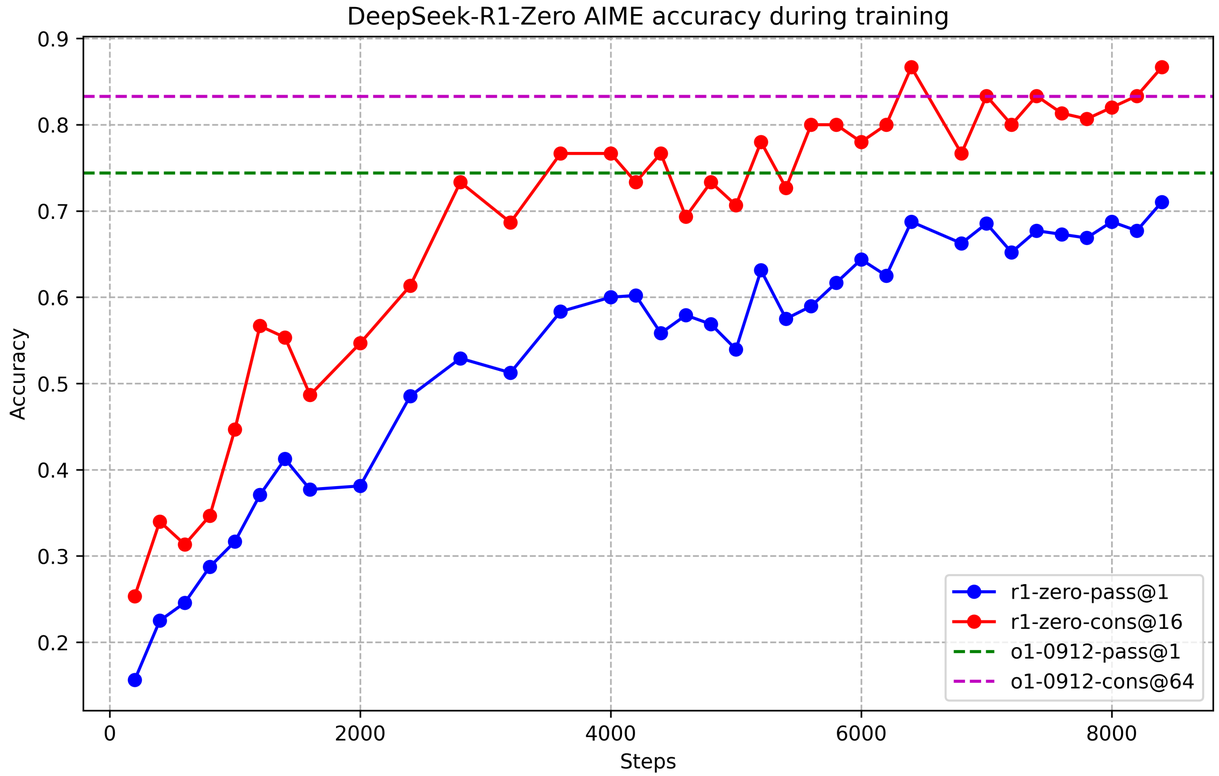
- In some domains, there's a clear right answer
  - e.g.: what's the derivative of $\ln(1 + e^{w \cdot x + b})$ with respect to w
  - or: where is the bug in

```
loss += 0.5 * (critic(state) - target)**2
optim.step()
loss.backward()
```

- If we can write a checker function to tell if we got the right answer, we can use this as feedback for RL
  - ***RL with verifiable rewards*** *(RLVR)*
- Can combine RLHF and RLVR: e.g., HF for being clear and looking nice; VR for correct answer, no LaTeX bugs

[DeepSeek-AI, 2025]

RLVR example: DeepSeek-R1-Zero



- RLVR applied directly to base transformer language model (pre-trained only, no SFT)
  - reward for correctness of final answer and for formatting
- First answer's score on AIME (pass@1) rose from poor (~15%) to pretty good (~71%). Majority vote of 16 answers winds up beating competing OpenAI model

Reasoning models

- For models that write code, solve math problems, etc., it's common to give them "scratch paper"
  - first generate a strategy into a ***reasoning buffer*** (still a string of tokens, but invisible to user)
  - then work through the solution step by step in this buffer
  - and finally generate the user-visible solution into an output token buffer
- Can use SFT to provide examples of how to reason, then RLVR to make reasoning more accurate
- Side effect: enables ***test-time scaling***
  - we can afford more iterations of RL (and promote efficient reasoning) if we limit size of reasoning buffer in training
  - at test time we can spend more tokens on the reasoning buffer to get better results (up to some ceiling)

Compute

take 10-405/605/805!

- Lots of clever optimizations go into training and serving a large model
  - model architecture: MoE, GQA
  - split data and compute across multiple GPUs, servers (e.g., in MoE, split experts to different devices)
  - compute attention matrix in blocks of rows (FlashAttention)
  - be careful of memory: mixed precision, rematerialization (checkpointing), gradient accumulation
  - GPU kernel engineering (e.g., fusing multiple steps)

Reuse

- Attention on long contexts is expensive!
  - big matrix multiplies: saturates memory bandwidth
- But the exact same computations show up multiple times in real workloads → can save and reuse results
  - Example 1: many users ask the exact same question (same prompt) — or a refinement of the same question (one prompt is a prefix of another)
  - Example 2: through the course of a conversation, each prompt has previous prompt as a prefix
  - Example 3: we have 1000 token prompt and 300 token response; response token 301 needs all the same matrix computations as token 300 did (plus some more)

Caching

- Treat as a caching problem: save results in multiple levels of storage
  - each level larger but slower than previous one
  - when a level gets full, evict (bump down to slower storage) the results that we are least likely to reuse
  - or drop entirely once largest/slowest level fills up
- Unit of reuse: (key, value) pairs at each attention layer
  - expensive: needed forward pass up to that layer
  - worth saving: future tokens attend over past keys/values
  - note: past queries aren't reused (generating a new token needs *this token's* query and *past tokens'* keys/values)
- Unlike some caches, can't evict independently: attention needs all previous tokens to generate next

KV cache and prefix cache

- Two important cases:
  - ***KV cache**:* previous tokens in the current generation
  - ***prefix cache**:* previous generations
- Often share storage in a single caching system, but different properties (e.g., mutability), so different treatment

Prefill vs generation

- Two distinct workloads:
  - ***prefill****:* process a long context
  - ***generation****:* produce new tokens one at a time
- Prefill is much easier to parallelize: tokens are all known ahead of time, no need to complete computation for position i before starting to process position $i+1$
  - e.g., add a new document to our database for RAG
  - or process a document for classification (e.g., sentiment of a review)
- Generation seems inherently sequential, but …

Speculative decoding

the mayor of New

- To improve latency of generation (at some cost to total work), we'd like to generate multiple tokens at once
- Idea: generate a few tokens (say, 5–10) from a cheaper model — this can be extremely fast
  - cheaper model will often be right, especially in easy cases (like filling in boilerplate arguments to a function)
- Then verify these tokens in parallel w/ expensive model
  - i.e., generate each position conditioned on previous positions being correct
  - keep an initial segment of tokens that agree — once the simpler model makes a mistake, have to discard all future tokens
- We win as long as simpler model is right often enough

Speculative decoding

the mayor of New York City, Zohran Mamdani, appeared at an event

- To improve latency of generation (at some cost to total work), we'd like to generate multiple tokens at once
- Idea: generate a few tokens (say, 5–10) from a cheaper model — this can be extremely fast
  - cheaper model will often be right, especially in easy cases (like filling in boilerplate arguments to a function)
- Then verify these tokens in parallel w/ expensive model
  - i.e., generate each position conditioned on previous positions being correct
  - keep an initial segment of tokens that agree — once the simpler model makes a mistake, have to discard all future tokens
- We win as long as simpler model is right often enough

Speculative decoding

York City, Zohran Mamdani, signed at an event

the mayor of New York City, Zohran Mamdani, appeared at an event

- To improve latency of generation (at some cost to total work), we'd like to generate multiple tokens at once
- Idea: generate a few tokens (say, 5–10) from a cheaper model — this can be extremely fast
  - cheaper model will often be right, especially in easy cases (like filling in boilerplate arguments to a function)
- Then verify these tokens in parallel w/ expensive model
  - i.e., generate each position conditioned on previous positions being correct
  - keep an initial segment of tokens that agree — once the simpler model makes a mistake, have to discard all future tokens
- We win as long as simpler model is right often enough

Language model as a classifier

- LLMs provide a highly informative prior for supervised learning tasks: trillions of tokens of human knowledge
- Suppose we can afford just 10 labeled examples in a new classification task; can we harness the LLM's prior to generalize well?

Representing examples for the LLM

- To take advantage of LLM's knowledge, need to transform (x, y) pairs into a format it can understand
- For inputs: write a short bit of code that turns features into natural language
  - e.g., `(red, 12)` → "the product is red and 12 cm tall"
- Optionally add a descriptive prompt:
  - "will it be a good gift for a kid?"
- For outputs:
  - pick appropriate tokens to represent the classes: e.g., "yes" and "no" for a binary classifier
  - to read out the answer, prohibit all other tokens: just test whether $P(\text{yes} \mid \text{input}) > P(\text{no} \mid \text{input})$

Fine-tuning

- First approach: fine-tuning
- Just like the fine-tuning examples above, continue training the pre-trained model on our new (tiny) dataset
  - use supervised loss (e.g., cross-entropy)
  - seems crazy that it can work with so few examples!
  - typically only need small learning rate and few epochs
- This variant is ***pattern-based*** fine-tuning
  - for small datasets, works better than other FT variants such as adding a new prediction head to the network

In-context learning

- Second approach: in-context learning
- Just include training examples in prompt before asking about test example:

```
red, 12 cm tall? Yes
blue, 15 cm tall? No
. . .
orange, 9 cm tall?
```

training examples

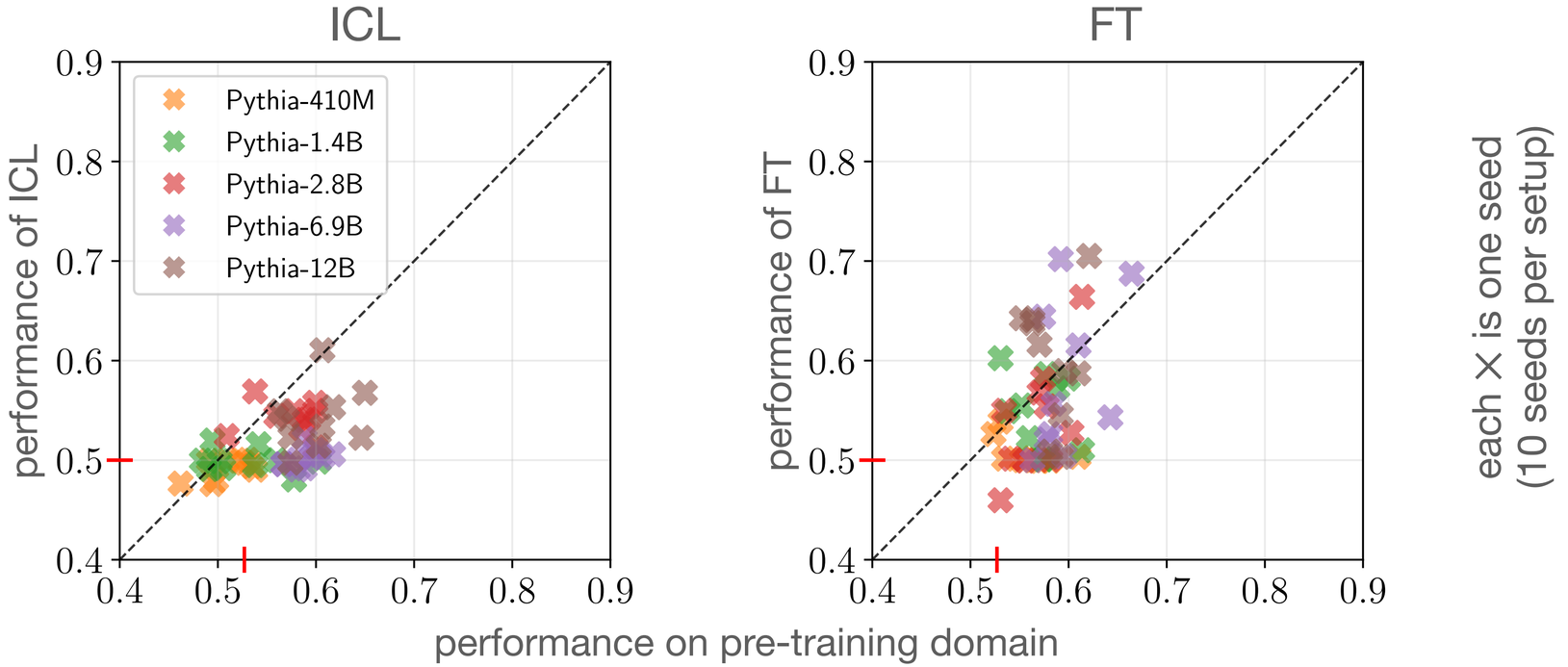
test input

- As above, read answer as $P(\text{yes} \mid \text{input}) > P(\text{no} \mid \text{input})$
- Huge advantage: no training needed!
  - fast, convenient
  - no need to know weights (e.g., API-only access)

[Mosbach et al., 2023]

Comparison

Graphs: 16 training examples, textual entailment task, models of multiple sizes with the same data (the Pile) and training pipeline



- ICL is fast and convenient, and can perform well for small training set sizes
- FT tends to win as we get more training data
- Both do better when base model is better — crossover point depends on base model
  - the better the model, the longer ICL remains competitive